

Il linguaggio Prolog

Intelligenza Artificiale e Laboratorio

Gian Luca Pozzato

Introduzione

- Prolog: **programming** in **logic**
- Linguaggio dichiarativo
- non descrivo *cosa* fare per risolvere un problema
- descrivo la situazione reale con **fatti** e **regole** e chiedo all'interprete di verificare se un **goal** segue oppure no secondo la logica classica
- molto utilizzato in AI

```
1  /* imports */
2  :- use_module(library(lists)).
3  :- op(300,fy,!),
4  op(500,xfy,>=>),
5  op(400,xfy,>=>),
6  op(400,xfy,^),
7  op(400,xfy,v).
8
9
10 prova(Formula):-prove([Formula],Tree),write(Tree),!.
11 prova(_):-write('Not Valid').
12
13
14 fillTheHole(NS,ListToFill,NewNS):-select(hole,NS,TempNS),!,
15                                 append(TempNS,ListToFill,NewNS).
16 fillTheHole(NS,ListToFill,[[[Fml,NewGamma],AppliedConditionals]|TempNS]):-select([Fml,Gamma],AppliedConditionals,NewGamma),
17                                 fillTheHole(Gamma,ListToFill,NewGamma).
18
19 selectList([],NS,[hole|NS]).
20 selectList([Fml|Tail],NS,NewNS):-select(Fml,NS,TempNS),
21                                 selectList(Tail,TempNS,NewNS).
22
23 deepSelect(List,NS,NewNS):-selectList(List,NS,NewNS).
24 deepSelect(List,NS,[[[Fml,NewGamma],AppliedConditionals]|NewNS]):-select([Fml,Gamma],AppliedConditionals,NewGamma),
25                                 deepSelect(List,NewGamma,NewNS).
26
27 memberList([],_).
28 memberList([Head|Tail],NS):-member(Head,NS),memberList(Tail,NS).
29 deepMember(List,NS):-memberList(List,NS).
30 deepMember(List,NS):-member([_,Gamma],_,NS),
31                           deepMember(List,Gamma).
32
33
34
35 prove(NS,tree(ax,NS,no,no,no)):-deepMember([P,!P],NS),!.
36 prove(NS,tree(axt,NS,no,no,no)):-deepMember([!top],NS),!.
37 prove(NS,tree(axb,NS,no,no,no)):-deepMember([!bot],NS),!.
38 prove(NS,tree(neg,NS,SubTree,no,no)):-deepSelect([!A],NS,NewNS),!,fillTheHole(NewNS,[A],DefNS),prove(DefNS,SubTree).
39 prove(NS,tree(or,NS,SubTree,no,no)):-deepSelect([A v B],NS,NewNS),!,fillTheHole(NewNS,[A,B],DefNS),prove(DefNS,SubTree).
40 prove(NS,tree(andn,NS,SubTree,no,no)):-deepSelect([!(A ^ B)],NS,NewNS),!,fillTheHole(NewNS,[!A,!B],DefNS),prove(DefNS,SubTree).
41 prove(NS,tree(imp,NS,SubTree,no,no)):-deepSelect([A => B],NS,NewNS),!,fillTheHole(NewNS,[A,B],DefNS),prove(DefNS,SubTree).
42 prove(NS,tree(condp,NS,SubTree,no,no)):-deepSelect([A => B],NS,NewNS),!,fillTheHole(NewNS,[A,B],DefNS),prove(DefNS,SubTree).
43 prove(NS,tree(orn,NS,Sub1,Sub2,no)):-deepSelect([!(A v B)],NS,NewNS),!,fillTheHole(NewNS,[A],NS1),fillTheHole(NewNS,[B],NS2),
44 prove(NS,tree(orp,NS,Sub1,Sub2,no)):-deepSelect([A ^ B],NS,NewNS),!,fillTheHole(NewNS,[A],NS1),fillTheHole(NewNS,[B],NS2),
45 prove(NS,tree(impn,NS,Sub1,Sub2,no)):-deepSelect([!(A => B)],NS,NewNS),!,fillTheHole(NewNS,[A],NS1),fillTheHole(NewNS,[B],NS2),
46 prove(NS,tree(condn,NS,Sub1,Sub2,Sub3)):-
47     deepSelect([!(A => B)],NS,NewNS),!,
48     \+member([A => B],[C,Delta],AppliedConditionals),!,
49     prove([A,!C],Sub2),
50     prove([C,!A],Sub3),
51     fillTheHole(NewNS,[!(A => B)],[[[B|Delta],[(A => B)|AppliedConditionals]]],DefNS),
52     prove(DefNS,Sub1).
53
54
55
56 /* Auxiliary predicates for building the latex source file containing a sequent */
57 getRuleName(tree(ax,_,_,_,_)):-write('AX'),!.
```

Per cominciare

- Rappresentiamo con dei **fatti** un dominio di interesse
- Esempio:
 - fatti per descrivere che un alimento contiene più calorie rispetto ad un altro
 - `piuCalorico(wurstel,banana)`
 - rappresenta il fatto che il würstel è un alimento maggiormente calorico rispetto alla banana

Per cominciare

- Rappresentiamo con **regole** possibili inferenze
- Regola:

`head :- subgoal1, subgoal2,..., subgoaln`

- Esempio:

`felino(X) :- gatto(X)`

- rappresenta la regola che permette di concludere che i gatti sono felini

E allora?

- Proviamo!

Idee di base

- Regole ricorsive
- L'interprete analizza fatti e regole nell'ordine in cui si trovano nel programma
- Meccanismo di pattern matching per unificare variabili e termini
- L'interprete, dato un programma, cerca di dimostrare un goal considerando fatti e applicando regole, nel secondo caso generando sotto-goal

Definizioni

- Fatti e regole contengono:
 - Atomi
 - costanti
 - *Esempi:* verza, 'Questo è un atomo', a, anche_questo_atomo
 - numeri
 - Variabili (iniziale maiuscola)
 - Termini composti, ottenuti applicando un funtore a termini (atomi, variabili, altri funtori)
 - *Esempi:* sopra(penna,tavolo), sopra(penna,X), nato(giovanni, data(torino, '12/07/1995'))

Definizioni

- Fatti e regole => Clausole
 - Fatto: predicato seguito da un punto
 - Regola costituita da testa e corpo `testa :- corpo`
 - testa: predicato
 - corpo: sequenza di predicati separati da virgola
- Programma Prolog = insieme finito di clausole
- L'interprete Prolog compila/consulta un programma e viene interrogato con un goal

Lista

- Struttura dati principale in Prolog
- Rappresentazione esplicita: termini fra parentesi quadre separati da virgole
 - *Esempio:* [1, ciao, nato(giovanni, torino, 2/3/1991), 2]
- Lista vuota: []
- Gli elementi di una lista sono termini o altre liste
 - *Esempio:* [23, [], ciao, [21, saluti], 11]

Lista

- Una lista è caratterizzata da una testa e da una coda
 - testa: primo termine (a sinistra) della lista
 - coda: la lista dei termini dal secondo (incluso) in poi
- Rappresentata come [Head | Tail]
- Utile per manipolare le liste

Lista

```
?- [1,2,3,4,5] = [Head | Tail].
```

```
Head = 1
```

```
Tail = [2,3,4,5] = [Head | Tail]
```

```
Yes
```

```
?- [a, ciao, [], 2, [1, saluti]] = [Head | Tail].
```

```
Head = a
```

```
Tail = [ciao, [], 2, [1, saluti]]
```

```
Yes
```

Lista

- Alcuni utili predicati built-in:
 - `length(Lista,N)`
 - ha successo se la lista `List`a contiene `N` elementi
 - `member(Elemento,List)`
 - ha successo se la lista `List`a contiene il termine `Elemento`
- ovviamente, si possono usare le variabili...

Lista

- Operazioni di manipolazione di liste mediante clausole ricorsive
- Definizione ricorsiva basata su Head e Tail
- Esempio: lista di interi, voglio fare la somma degli elementi
 - Base: se la lista è vuota la somma è 0
 - Passo: se la lista è [Head|Tail], la somma è la somma di Head con la somma (calcolata invocando ricorsivamente il predicato) degli elementi di Tail

Lista

- Operazioni utili:
 - select di un elemento: come member, ma rimuovendo dalla lista iniziale l'elemento cercato
 - inversione della lista
 - concatenamento di liste